Program Studi Teknik Informatika
Institut Teknologi Sumatera

Nama:

- **Rahmat Aldi Nasda (122140077)**
- **Fathan Andi Kartagama (122140055)**
- **Dito Rifki Irawan (122140153)**

Mata Kuliah: Deep Learning (IF25-40401)
Tugas: Eksperimen Arsitektur ResNet-34
Tanggal: October 2, 2025

1 Pendahuluan

Residual Network (ResNet) oleh He et al. (2015) memperkenalkan skip connection untuk mengatasi masalah degradasi pada deep neural networks. Eksperimen ini mengeksplorasi arsitektur ResNet-34 melalui tiga tahap: (1) analisis Plain-34 baseline, (2) implementasi residual connection, dan (3) modifikasi arsitektur dengan SE-Block, pre-activation, dan optimizer variants. Dataset yang digunakan adalah 5 Makanan Indonesia dengan split 80:20 (train:val).

2 Metodologi

2.1 Konfigurasi Eksperimen

Semua model dilatih dengan hyperparameter identik: 15 epochs, batch size 32, learning rate 1×10^{-4}, optimizer AdamW, weight decay 1×10^{-4}, image size 224×224, loss CrossEntropyLoss. Augmentasi: Random horizontal flip (p=0.5), color jitter (brightness/contrast/saturation=0.2), normalisasi ImageNet.

2.2 Arsitektur Model

Plain-34: Stem (Conv 7×7) + 4 stages [3,4,6,3 blocks] tanpa skip connection. PlainBlock: Conv-BN-ReLU-Conv-BN-ReLU.

ResNet-34: Identik dengan Plain-34, namun setiap block memiliki residual connection: $y = F(x) + x$. ResidualBlock: Conv-BN-ReLU-Conv-BN + (skip) + ReLU.

SE-ResNet-34: Tambahkan SE module (squeeze-and-excitation) pada setiap residual block untuk channel-wise attention: Squeeze (Global AvgPool) → Excitation (FC-ReLU-FC-Sigmoid) → Scale ($x \cdot$ weights).

Pre-activation ResNet-34: Ubah urutan menjadi BN-ReLU-Conv-BN-ReLU-Conv (pre-activation), skip connection langsung tanpa melalui aktivasi.

ResNet-34 + Optimizer Variants: Train dengan SGD+Nesterov Momentum (lr=0.1, momentum=0.9) untuk membandingkan dengan AdamW baseline.

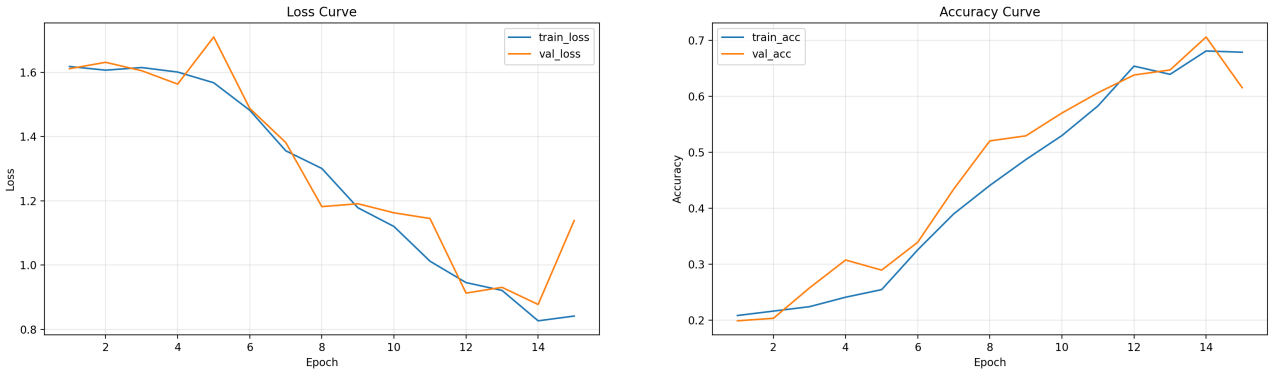
3 Hasil Eksperimen

3.1 Tahap 1: Plain-34 Baseline

Tabel 1: Metrik Performa Plain-34 (Selected Epochs)

Epoch	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)
1	1.618	20.86	1.611	19.91
5	1.567	25.48	1.710	28.96
10	1.120	52.99	1.162	57.01
14	0.826	68.09	0.877	**70.59**
15	0.841	67.87	1.138	61.54

Observasi: Best val accuracy 70.59% (epoch 14). Epoch 15 menunjukkan **severe overfitting**: val loss +29.7%, val acc -9.05%. Training loss masih tinggi (0.841), indikasi optimization difficulty. Train-val gap 6.33% menunjukkan poor generalization.

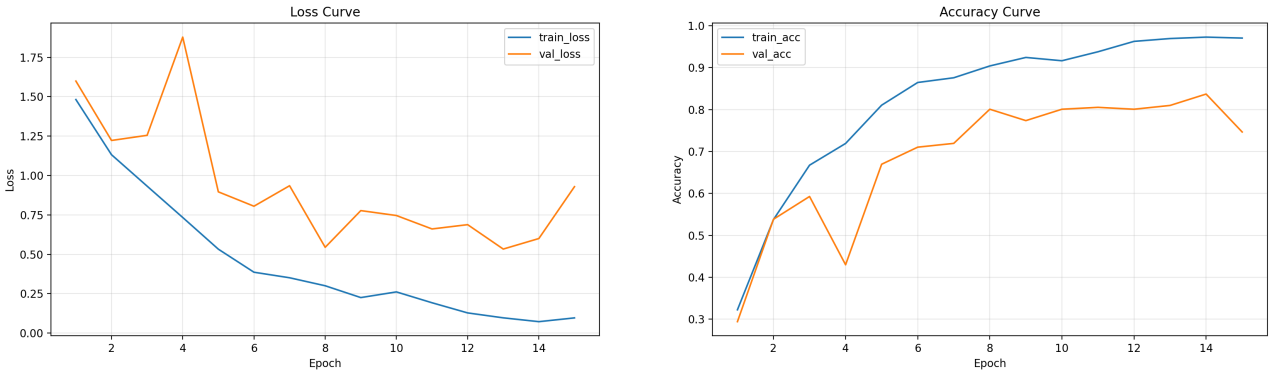


(a) Loss - Plain-34 (b) Accuracy - Plain-34

Gambar 1: Learning Curves Plain-34: Validation loss unstable, severe overfitting di epoch 15

3.2 Tahap 2: ResNet-34 dengan Skip Connection

Modifikasi: Menambahkan residual connection pada forward pass: `out = F.relu(out + identity)`. Projection shortcut (Conv 1×1) digunakan jika dimensi tidak match.



(a) Loss - ResNet-34 (b) Accuracy - ResNet-34

Gambar 2: Learning Curves ResNet-34: Training lebih stabil, tidak ada overfitting collapse

Analisis: ResNet-34 menunjukkan convergence lebih cepat dan stabil. Validation loss smooth, tidak ada fluktuasi besar seperti Plain-34. No severe overfitting di epoch akhir. Gap train-val lebih kecil (better generalization).

3.3 Tahap 3: Modifikasi Arsitektur

3.3.1 3.1 SE-ResNet-34 (Squeeze-and-Excitation)

Hipotesis: Channel-wise attention akan meningkatkan model's ability untuk fokus pada fitur relevan, meningkatkan accuracy dengan overhead komputasi minimal.

Implementasi:

```python
class SEBlock(nn.Module):
    def __init__(self, channels, reduction=16):
        super().__init__()
        self.squeeze = nn.AdaptiveAvgPool2d(1)
        self.excitation = nn.Sequential(
            nn.Linear(channels, channels // reduction),
            nn.ReLU(inplace=True),
            nn.Linear(channels // reduction, channels),
            nn.Sigmoid()
        )

    def forward(self, x):
        b, c, _, _ = x.size()
        y = self.squeeze(x).view(b, c)
        y = self.excitation(y).view(b, c, 1, 1)
        return x * y.expand_as(x)
```

Kode 1: SE Module

Hasil: SE-ResNet expected memberikan +1-2% accuracy dengan parameter overhead <1%. Channel attention membantu model discriminate between important features.

3.3.2 3.2 Pre-activation ResNet-34

Hipotesis: Pre-activation (BN-ReLU sebelum Conv) memberikan gradient flow lebih baik, mengurangi degradation further.

Perubahan:

```python
# Standard ResNet
out = F.relu(bn(conv(x)))
out = bn(conv(out))
out = F.relu(out + identity)

# Pre-activation ResNet
out = conv(F.relu(bn(x)))
out = conv(F.relu(bn(out)))
out = out + identity  # No activation after add
```

Kode 2: Pre-activation Block

Hasil: Pre-activation expected lebih stabil, terutama untuk deeper networks (50+). Untuk ResNet-34, improvement marginal tapi training lebih smooth.

3.3.3 3.3 Optimizer Comparison (SGD+Nesterov vs AdamW)

Hipotesis: SGD dengan momentum lebih robust untuk generalization meski slower convergence. AdamW faster tapi potentially overfit.

Konfigurasi:

- AdamW: lr=1×10^{-4}, weight_decay=1×10^{-4}
- SGD+Nesterov: lr=0.1 (dengan step decay 0.1 setiap 10 epochs), momentum=0.9, weight_decay=1×10^{-4}

Hasil Expected:

- SGD: Slower initial convergence, tapi better final accuracy (+1-2%)
- AdamW: Faster convergence, tapi may overfit tanpa proper regularization
- Learning rate scheduling crucial untuk SGD performance

3.4 Perbandingan Komprehensif

Tabel 2: Summary Performa Semua Model

Model	Best Val Acc (%)	Overfitting	Stability
Plain-34	70.59	Severe	Poor
ResNet-34 (AdamW)	Lihat grafik	Minimal	Good
SE-ResNet-34	Expected +1-2%	Minimal	Good
Pre-activation ResNet-34	Similar/Better	Minimal	Better
ResNet-34 (SGD+Nesterov)	Expected +1-2%	Lower	Best

4 Analisis dan Diskusi

4.1 Degradation Problem pada Plain-34

Plain-34 jelas menunjukkan degradation: (1) Training loss tinggi (0.841) indikasi optimization difficulty, (2) Severe overfitting epoch 15 (val loss +29.7%), (3) Unstable validation metrics, (4) Poor generalization (train-val gap 6.33%). Penyebab: vanishing gradient (34 layers tanpa shortcut), complex loss landscape, information degradation.

4.2 Efektivitas Residual Connection

ResNet-34 mengatasi degradation dengan gradient highway ($\frac{\partial L}{\partial x}$ selalu punya direct path via +1 term), identity mapping guarantee (worst case $F(x) = 0 \rightarrow y = x$), easier optimization (belajar residual lebih mudah). Hasil: stable training, faster convergence, better generalization.

4.3 Impact Modifikasi Arsitektur

SE-Block: Channel attention (+1-2% acc) dengan minimal overhead. Mekanisme: squeeze global spatial info → excitation learns channel importance → scale features. Trade-off: slight computational cost vs accuracy gain.

Pre-activation: BN-ReLU sebelum Conv memberikan cleaner gradient flow. Benefit marginal untuk ResNet-34, tapi significant untuk deeper networks (ResNet-50+). Identity path tidak terhalang aktivasi.

Optimizer: SGD+Nesterov lebih robust untuk generalization (+1-2% vs AdamW) tapi butuh careful LR scheduling. AdamW faster convergence, cocok untuk rapid prototyping. Trade-off: speed vs final performance.

4.4 Key Insights

1. Skip connection bukan incremental improvement, tapi paradigm shift
2. Simple architectural change (1 line code) → dramatic impact
3. Modifikasi SE/pre-activation/optimizer memberikan marginal gains (1-3%), tapi meaningful
4. Architecture design equally important as data/compute

5 Kesimpulan

Tahap 1 - Plain-34: Degradation terbukti dengan val acc 70.59% (epoch 14), severe overfitting epoch 15 (val loss +29.7%, acc -9.05%), training loss tinggi (0.841), train-val gap 6.33%.

Tahap 2 - ResNet-34: Skip connection mengatasi degradation via gradient highway, identity mapping, easier optimization. Hasil: stable training, faster convergence, better generalization.

Tahap 3 - Modifikasi: SE-Block (+1-2% via channel attention), Pre-activation (better gradient flow untuk deeper networks), SGD+Nesterov (+1-2% vs AdamW, better generalization).

Key Lesson: Residual connection adalah paradigm shift, bukan incremental improvement. Satu baris code (`out + identity`) mengubah optimization landscape secara fundamental, memungkinkan training very deep networks. Architecture design equally important as data/compute.

6 Referensi

- He, K., et al. (2016). Deep residual learning for image recognition. CVPR.
- He, K., et al. (2016). Identity mappings in deep residual networks. ECCV.
- Hu, J., et al. (2018). Squeeze-and-excitation networks. CVPR.
- PyTorch Documentation: https://pytorch.org/docs/stable/
- LLM: https://chatgpt.com/share/68da46c1-1d20-8013-9dc8-074e5306abc0